FRONTEND DEVELOPMENT

Assignment 5

Project and team: BookingApp
Group: SE-2513
Members: Allazhar Aldongarov, Dias Serikov, Yerkenov Abylay

Objective

Extend our hotel booking website with JavaScript data handling, DOM updates, keyboard navigation, callbacks, sounds and event-triggered animations. Keep the code separated into readable HTML, CSS and JavaScript files.

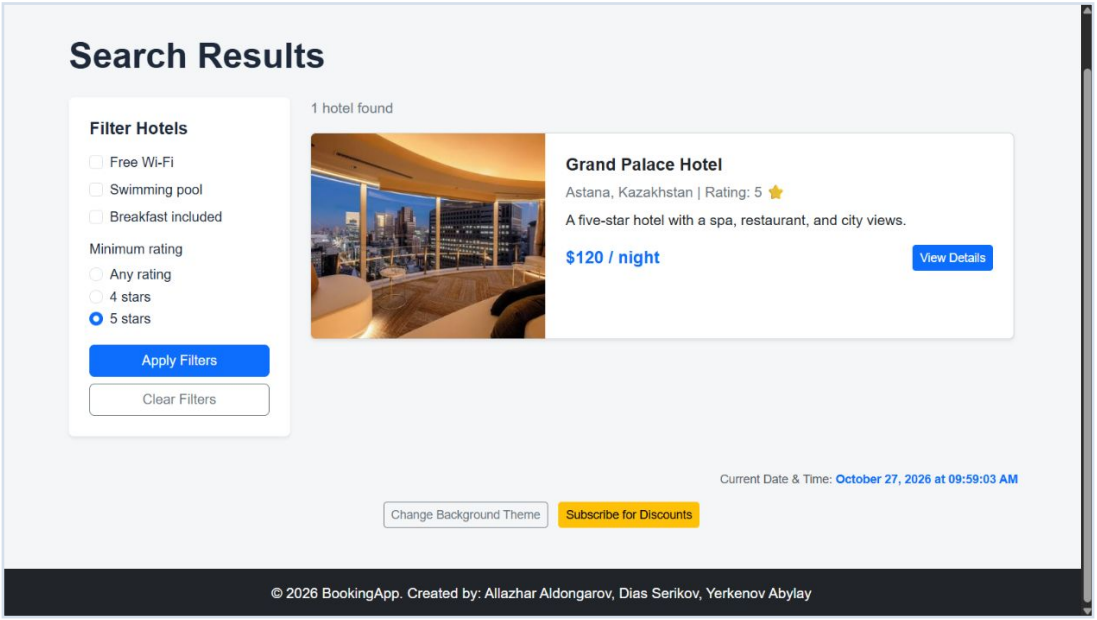
Requirement map

Assignment requirement	Project implementation
DOM selection and content updates	Hotel cards, filter results, booking review and service estimate
Dynamic styling and one attribute option	Background colour button and Hotel Details Read More
Button and keyboard events	Filter, form and estimator buttons; arrow-key menu navigation
Callbacks and switch statements	Multi-step booking callback; hotel rating switch
Objects, methods, arrays and loops	Hotel/service arrays, estimator methods and explicit for loops
Higher-order functions	filter, forEach and callback arguments
Sounds and animations	Single click audio instance; animated steps and results
Structure and user experience	Separate files, validation, focus handling and responsive layout

Assignment 4 features are retained. The teammate's dynamic hotel cards, rating filter and supplied sound are integrated with the new features. Forms are local demonstrations: no real booking, account, subscription or message is created.

Hotel cards and filters

1. Store the example hotels in an array of objects.
2. Read the selected rating and amenities when the filter form is submitted.
3. Use switch and filter to select matching hotels, then call renderHotels to rebuild the cards without a page reload.



Search results after applying a rating filter.

js/script.js

```
165             switch (rating) {
166                 case '4':
167                     filteredHotels = hotelsDatabase.filter(hotel => hotel.rating >= 4);
168                     break;
169                 case '5':
170                     filteredHotels = hotelsDatabase.filter(hotel => hotel.rating === 5);
171                     break;
172                 default:
173                     filteredHotels = hotelsDatabase;
174             }
175             filteredHotels = filteredHotels.filter(function (hotel) {
176                 return (!needsWifi || hotel.wifi) &&
177                     (!needsPool || hotel.pool) &&
178                     (!needsBreakfast || hotel.breakfast);
179             });
180             renderHotels(filteredHotels);
181         });
182         filterForm.addEventListener('reset', function () {
```

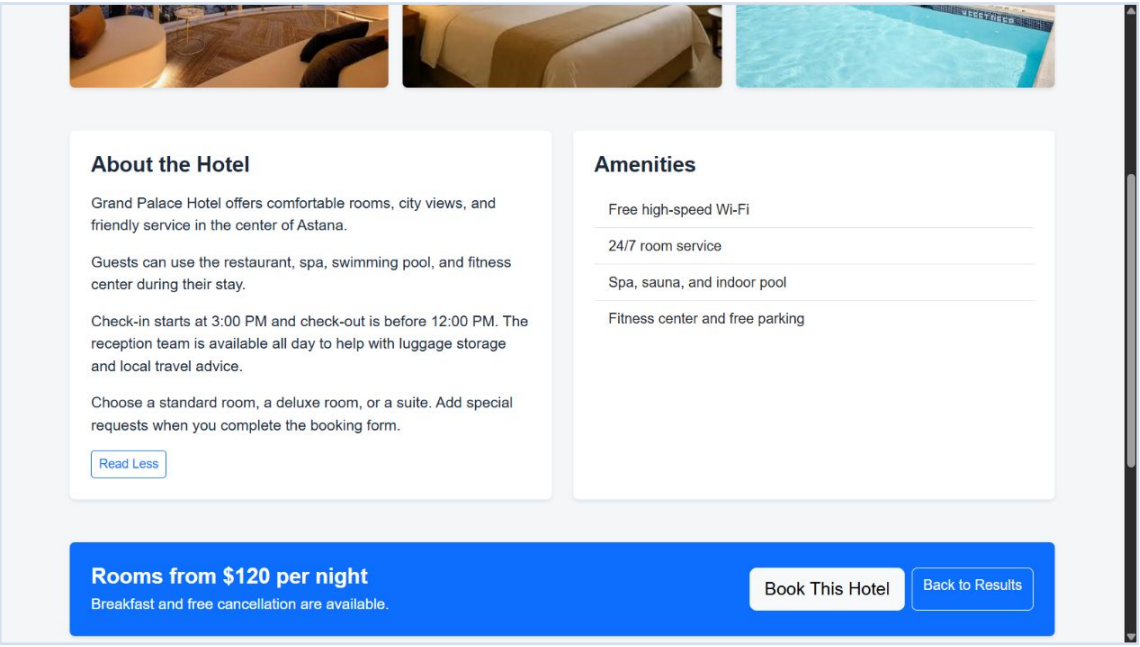
Code excerpt: js/script.js, lines 165-182.

How it works: DOM methods select the results container. forEach creates the cards, while filter returns a new array of matching objects. Selecting four stars means a minimum rating of four, so a five-star hotel also matches.

Demonstration: open Search, choose five stars and apply the filter. Change the rating or amenities and apply again. Use reset to restore the initial filters.

Read More and visual feedback

1. Keep extra hotel information in a separate element.
2. Toggle its visibility from the Read More button.
3. Update the button label and aria-expanded so the visual state and accessibility state agree.



Hotel Details with the additional description expanded.

js/booking.js

```
1    const readMoreButton = document.getElementById('hotel-read-more');
2
3    if (readMoreButton) {
4        const moreInformation = document.getElementById('hotel-more');
5
6        readMoreButton.addEventListener('click', function () {
7            const isOpen = readMoreButton.getAttribute('aria-expanded') === 'true';
8            moreInformation.style.display = isOpen ? 'none' : 'block';
9            moreInformation.setAttribute('aria-hidden', String(isOpen));
10           readMoreButton.setAttribute('aria-expanded', String(!isOpen));
11           readMoreButton.textContent = isOpen ? 'Read More' : 'Read Less';
12       });
13   }
14
15   // The identifier from Search chooses the same hotel on both pages.
16   const selectedHotel = getSelectedHotel();
17   document.querySelectorAll('a[href="booking.html"]').forEach(function (link) {
```

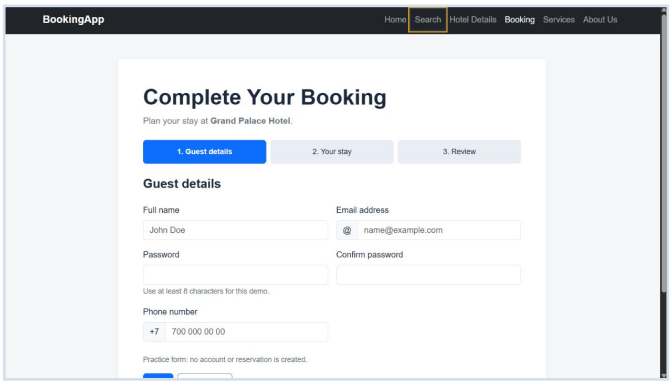
Code excerpt: js/booking.js, lines 1-17.

The Home and Search background button also demonstrates direct style changes. Existing FAQ answers expand with a CSS transition. New form and estimate states use event-triggered animation classes; animation rules stay in CSS.

Demonstration: expand and collapse the description, change the Home background, then open and close an FAQ answer.

Keyboard navigation and clicks

1. Select the navigation links.
2. Listen for keydown on the focused menu item.
3. Move focus with the arrow keys; Enter opens the focused link. Each page group includes this code in its existing JavaScript file. Button clicks restart one Audio instance per page so sounds do not overlap.



A menu item focused during keyboard navigation.

js/booking.js

```
250
251     for (let index = 0; index < menuLinks.length; index++) {
252         menuLinks[index].addEventListener('keydown', function (event) {
253             let nextIndex = index;
254
255             switch (event.key) {
256                 case 'ArrowRight':
257                 case 'ArrowDown':
258                     nextIndex = (index + 1) % menuLinks.length;
259                     break;
260                 case 'ArrowLeft':
261                 case 'ArrowUp':
262                     nextIndex = (index - 1 + menuLinks.length) % menuLinks.length;
263                     break;
264                 case 'Home':
265                     nextIndex = 0;
266                     break;
```

Code excerpt: js/booking.js, lines 250-266.

js/booking.js

```
278
279     const clickSound = new Audio('image/click.mp3');
280     clickSound.volume = 0.4;
281
282     document.addEventListener('click', function (event) {
283         const button = event.target.closest('button');
284         if (button && !button.disabled) {
285             clickSound.pause();
286             clickSound.currentTime = 0;
287             clickSound.play().catch(function () {
```

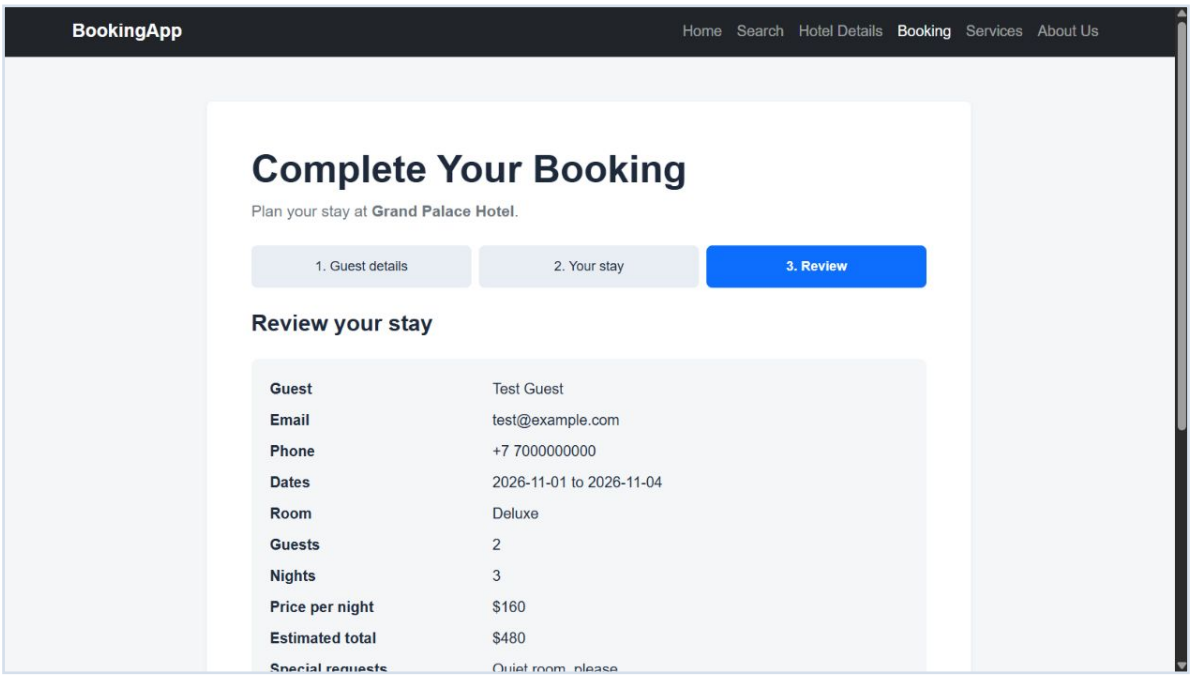
Code excerpt: js/booking.js, lines 278-287.

Demonstration: use Tab to reach a menu link, use the arrow keys, and press Enter. Click a button to trigger image/click.mp3. Audio output depends on the browser and device sound settings.

PART 1 - CALLBACKS AND STATE

Multi-step booking

1. Collect guest details, then stay details, then display a review.
2. Validate the current step before continuing.
3. Pass the step-display function as a callback. Next and Back change the visible step while the same input elements keep their values.



Booking review before the final demo confirmation.

js/booking.js

```
202
203     function changeStep(direction, onStepChange) {
204         const nextStep = currentStep + direction;
205         if (nextStep < 0 || nextStep >= steps.length) return;
206         if (direction > 0 && !validateStep(currentStep)) return;
207         clearErrors();
208         currentStep = nextStep;
209         onStepChange(currentStep);
210     }
211
212     nextButton.addEventListener('click', function () {
213         changeStep(1, showStep);
214     });
215
216     backButton.addEventListener('click', function () {
217         changeStep(-1, showStep);
218     });
```

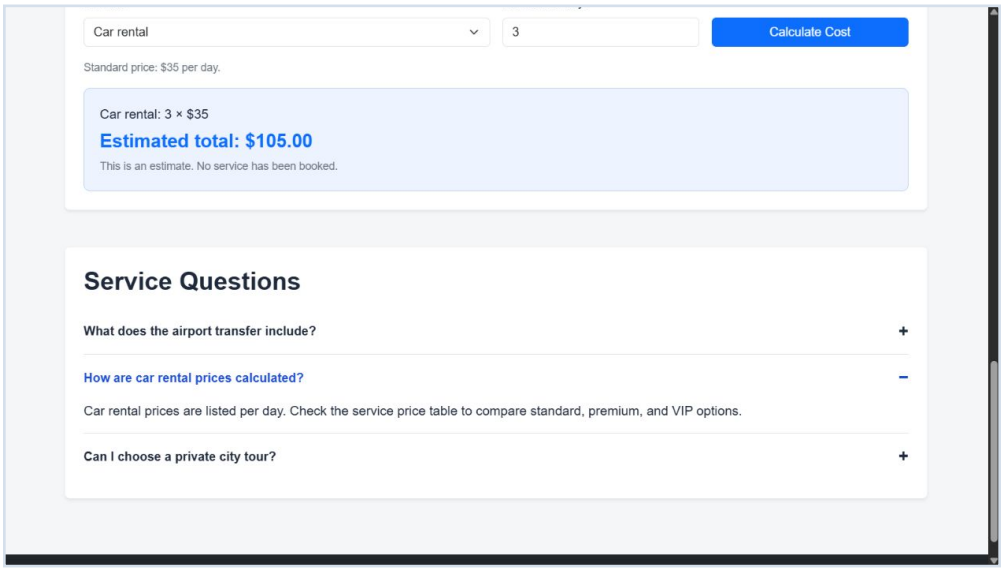
Code excerpt: js/booking.js, lines 202-218.

The booking summary uses object methods to calculate nights and total cost. Invalid input produces a nearby message and focused field. The final confirmation clearly states that no real booking or account was created.

Demonstration: try an invalid email, correct it, continue, go Back and check that the input is retained. Review and submit valid demonstration data.

Service price estimator

1. Store service names, rates and units in the services array.
2. Use a for loop to build the selection options.
3. Call serviceEstimator methods to find the chosen service and multiply its rate by the quantity. Display the result and trigger its CSS animation.



An estimated service price calculated from the selected service and quantity.

js/about-services.js

```
52  const serviceEstimator = {
53      getService(serviceId) {
54          return services.find(function (service) {
55              return service.id === serviceId;
56          });
57      },
58
59      calculateTotal(serviceId, quantity) {
60          const service = this.getService(serviceId);
61          return service.price * quantity;
62      }
63  };
64
65  const estimateForm = document.getElementById('service-estimate-form');
```

Code excerpt: js/about-services.js, lines 52-65.

js/about-services.js

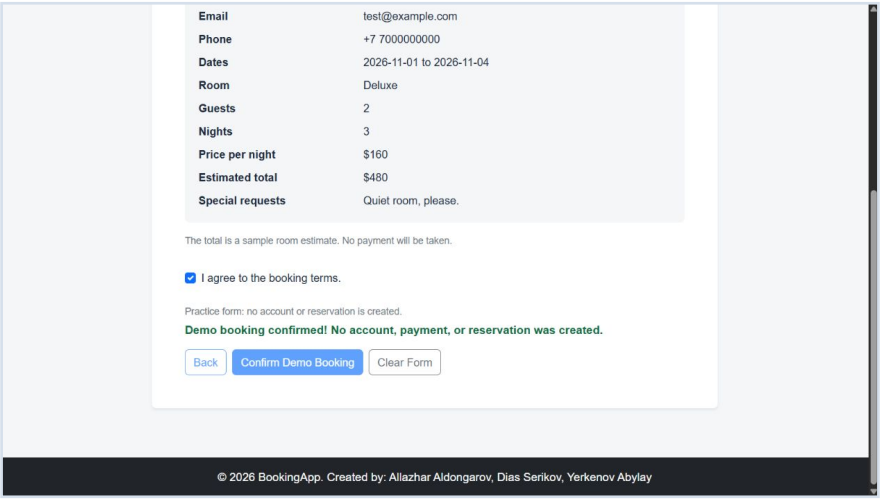
```
76      for (let i = 0; i < services.length; i++) {
77          const option = document.createElement('option');
78          option.value = services[i].id;
79          option.textContent = services[i].name;
80          serviceChoice.appendChild(option);
81      }
```

Code excerpt: js/about-services.js, lines 76-81.

Example: car rental at $35 per day for three days gives $105. The method uses this to call another method on the same object. No reservation or payment is made.

Project review

Pages	CSS and JavaScript
Home / Search	allstyle.css / script.js / hotels.js
Booking / Hotel Details	style.css / booking.js / hotels.js
About / Services	ablstyle.css / about-services.js



The local booking confirmation shown after valid demonstration input.

Verification and limitations

The screenshots record the local browser results. The website can be opened from index.html without a build step; Bootstrap is loaded from its existing CDN. Contact, subscription and booking forms are frontend demonstrations. There is no backend, payment processing or real delivery of messages.

Recorded checks: All six pages passed JavaScript syntax and local-reference checks. Browser checks covered filters, keyboard navigation, booking validation/back/reset and the $480 total, the $105 service estimate, popup, contact feedback, and 390px layouts.

Deployment

The website is deployed on Vercel: https://aishhko.vercel.app/.

Shared repository: https://github.com/jjestar/web
GitHub Pages: https://jjestar.github.io/web/

Reflection

The main challenge was keeping existing interactions while adding keyboard and sound behavior to each page group. Small functions and the original three groups of files make each feature easier to explain. We practised selecting DOM elements, handling events, preserving form state and using arrays and object methods for calculations.